

统计机器学习基础

Foundations of Statistical Machine Learning

——集成学习

Kai Chen

kaichen6@csu.edu.cn

24th September 2025



Overview

- 1 集成方法
- 2 AdaBoost 算法
- 3 AdaBoost 的泛化特性
- 4 提升与逻辑回归
- 5 梯度提升

弱学习器：应对偏差-方差权衡

定义 (弱学习)

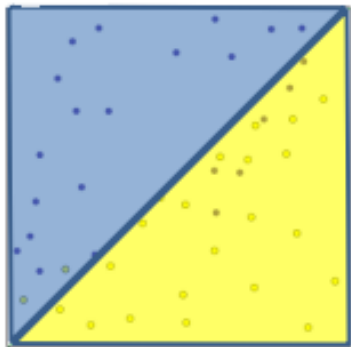
如果存在一个算法 A , $\gamma > 0$, 以及一个多项式函数 $\text{poly}(\cdot, \cdot, \cdot)$, 使得对于任何 $\epsilon > 0$ 和 $\delta > 0$, 对于所有 $\mathcal{X}$ 上的分布 D 以及任何目标概念 $c \in C$, 对于任何样本大小 $m \geq \text{poly}(1/\epsilon, 1/\delta, n, \text{size}(c))$, 以下成立:

$$\Pr_{S \sim D^m} \left[R(h_S) \leq \frac{1}{2} - \gamma \right] \geq 1 - \delta.$$

当这样的算法 A 存在时, 它被称为 C 的弱学习算法或弱学习器。弱学习算法返回的假设称为基分类器。

弱学习器

- 简单理解：一个模型，其性能仅比随机猜测略好，即其预测的准确率略高于50%（对于二分类问题），即为简单学习器（又称弱学习器）。例如朴素贝叶斯、逻辑回归、决策桩（或浅层决策树）



- 优点：低方差，通常不会过拟合

缺点：高偏差，无法解决困难的机器学习问题

为什么要提升弱学习器？

- 目标：自动分类呼叫请求类型（对方付费、电话卡、人对人……）

```
● yes I'd like to place a collect call long  
distance please (Collect)  
● operator I need to make a call but I need to  
bill it to my office (ThirdNumber)  
● yes I'd like to place a call on my master card  
please (CallingCard)
```

- 容易找到“经验法则”，这些法则“经常”是正确的。例如，如果话语中出现“card”，则预测为“calling card”
- 难以找到单个高准确率预测器

集成方法

- 组合多个预测变量以创建更准确的预测变量。
- 它们结合弱学习器返回的不同基本分类器以创建更准确的预测器。
-

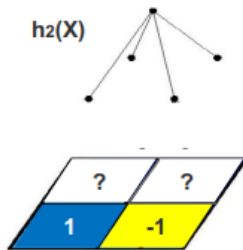
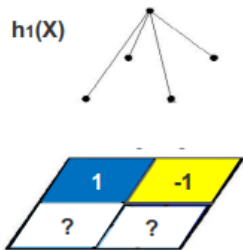
$$f(y|\mathbf{x}, \pi) = \sum_{m \in \mathcal{M}} w_m f_m(y|\mathbf{x})$$

其中 w_m 是可调参数。集成学习有时被称为委员会方法，因为每个基模型 f_m 都获得了一个加权的“投票”。

- 应该使用哪些基本分类器？
- 如何组合它们？

投票法

- 不学习单个（弱）分类器，而是学习多个在不同输入空间区域表现良好的弱分类器
- 输出类别：每个分类器的（加权）投票
- 最“确信”的分类器将以更大权重投票
- 分类器对特定空间区域最“确信”
- 平均而言，比单个分类器表现更好！



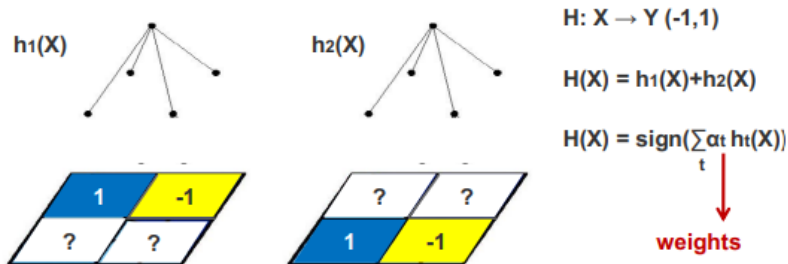
$$H: X \rightarrow Y (-1,1)$$

$$H(X) = h_1(X) + h_2(X)$$

$$H(X) = \text{sign}\left(\sum_t \alpha_t h_t(X)\right)$$

↓
weights

投票法



- 强制分类器 h_t 学习输入空间的不同区域？
- 对不同分类器的投票进行加权 a_t ？
 - 硬投票 (Hard Voting): 对于分类问题, 选择得票最多的类别作为最终预测。
 - 软投票 (Soft Voting): 对于分类问题, 计算每个类别的平均概率; 对于回归问题, 计算预测值的平均值。

- 训练多个不同的学习器，并将它们的预测结果作为新特征输入到一个元学习器（meta-learner）中。
- 元学习器学习如何最佳地组合这些预测结果以得到最终预测。

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \sum_{i=1}^N L(y_i; \sum_{m=1}^M w_m f_m(\mathbf{x}))$$

希望通过调整权重 w_m 来最小化损失函数 L 。这里 $f_m(x)$ 是第 m 个模型对输入 x 的预测, y_i 是实际的输出。

Bagging ((Bootstrap Aggregating))

- 回忆决策树

- 优点：可解释、能处理离散和连续特征、对异常值鲁棒、低偏差等
- 缺点：高方差

- 树是集成方法的完美候选

- 考虑对许多（几乎）无偏的树估计器进行平均（回归问题）或多数投票（分类问题）。
- 偏差保持相似，但方差降低

- 这称为 Bagging (Bootstrap Aggregating, 自助聚合) (Breiman,1996)
通过自助采样 (Bootstrap) 从原始数据集中有放回地重复抽样，生成多个子数据集，在每个子数据集上训练一个学习器，然后取平均

$$f(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$$

Bootstrap: 统计学上“掷 n 面骰子 n 次”的术语

典型的 Bagging 实现是随机森林 (Random Forest)。通过引入随机性降低树之间的相关性

- 对于 $b = 1, \dots, B$,
 - ① 绘制一个自助数据集 Z^* 。
 - ② 在 Z^* 上学习一棵树 $f_b(\cdot)$ ，特别是从 p 个特征中随机选择 m 个特征作为分裂前的候选。
- 输出：
 - 回归: $f(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$
 - 分类: 多数投票
- 通常取 $m \leq \sqrt{p}$

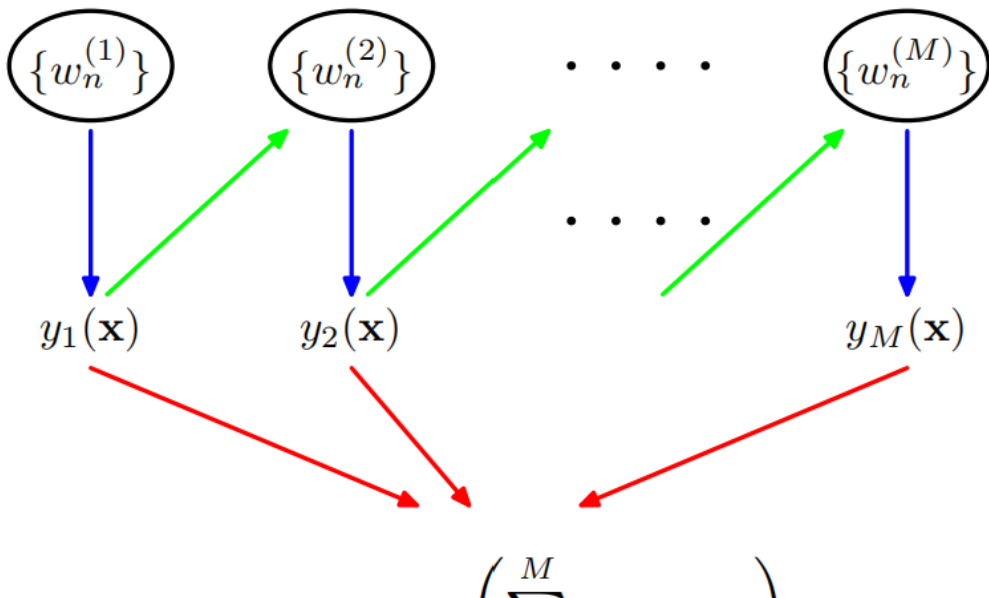
提升 [Schapire'89]

- 思想：给定一个弱学习器，多次在（重加权）训练数据上运行它，然后让学习到的分类器投票
- 在每次迭代 t ：
 - 根据分类错误的程度对每个训练样本加权
 - 学习一个弱假设 h_t
 - 为该假设赋予一个强度 a_t
- 最终分类器：

$$H(X) = \text{sign}\left(\sum a_t h_t(X)\right)$$

- 典型的 Boosting 实现包括 AdaBoost、Gradient Boosting 和 XGBoost。
- 重要问题：
 - 我们优化的准则是什么？（损失度量）
 - 我们希望以相同方式估计每个新组件分类器（模块化）

提升



Outline

- 1 集成方法
- 2 AdaBoost 算法
- 3 AdaBoost 的泛化特性
- 4 提升与逻辑回归
- 5 梯度提升

AdaBoost 算法

- 在第 k 次迭代中，我们找到（任意）分类器 $h(\mathbf{x}; \theta_k^*)$ ，使得加权分类误差

$$\mathcal{E}_k = \frac{\sum_{i=1}^n W_i^{k-1} I(y_i \neq h(\mathbf{x}_i; \theta_k^*))}{\sum_{i=1}^n W_i^{k-1}}$$

该方法优于随机猜测。

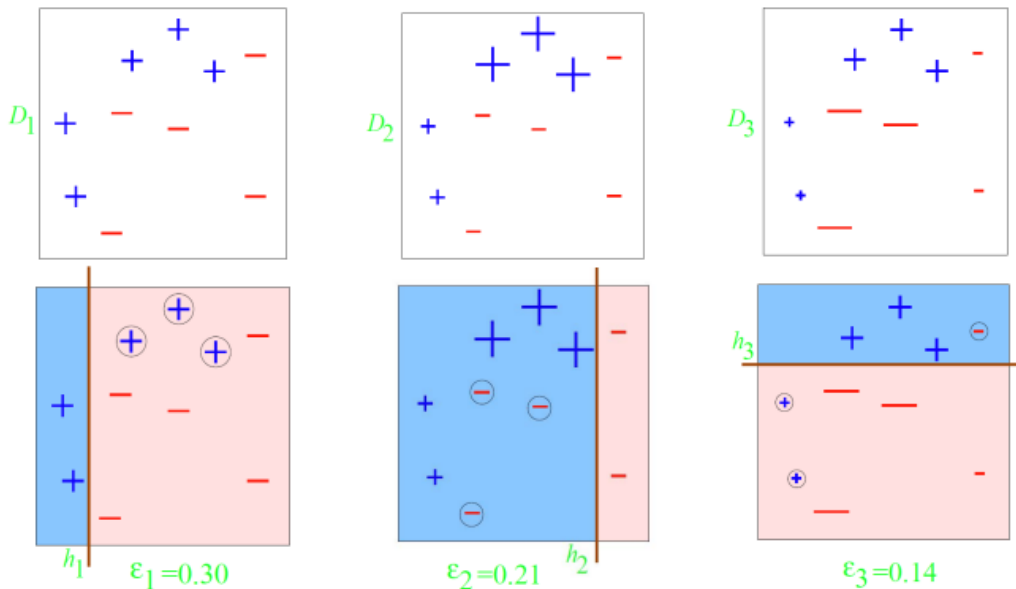
- 这应该“容易”——使用弱分类器
- 确定给新组件分类器分配多少“投票”：

$$a_k = 0.5 \log \frac{1 - \mathcal{E}_k}{\mathcal{E}_k}$$

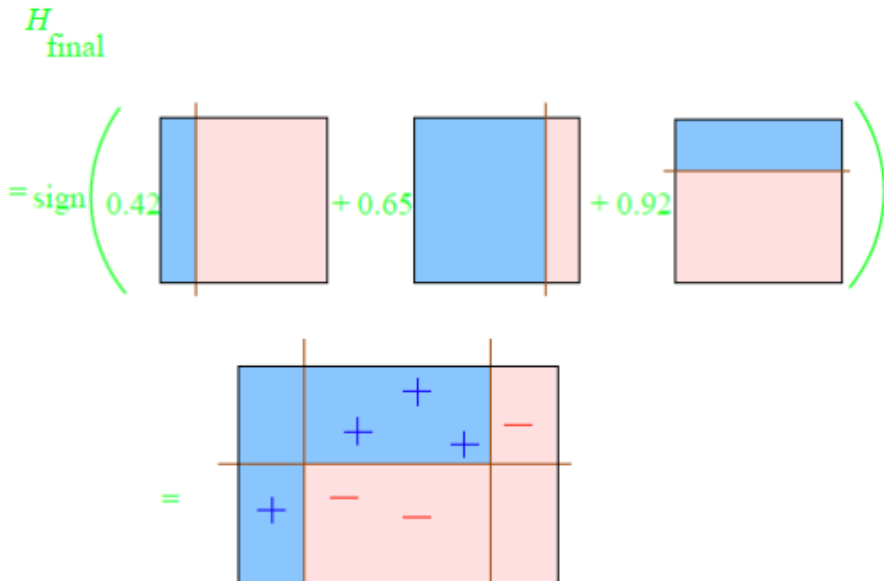
- 更强的分类器获得更多投票
- 更新训练样本权重：

$$W_i^k = W_i^{k-1} \exp\{-y_i a_k h(\mathbf{x}_i; \theta_k)\}$$

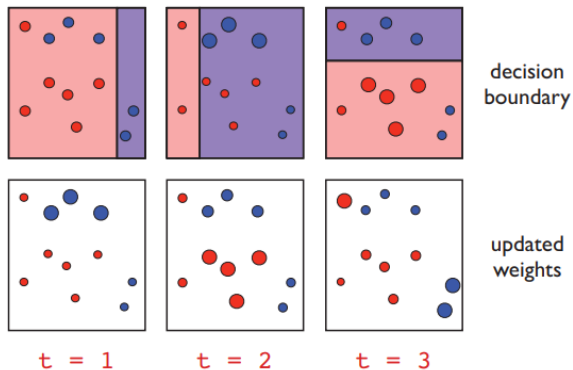
提升示例（决策桩）



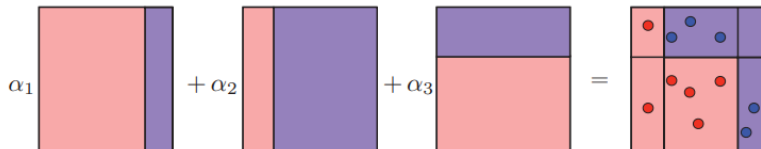
提升示例（决策桩）



AdaBoost 算法示例



(a)



- 实践中使用的弱学习器：
 - 决策桩（轴平行分裂）
 - 决策树（例如 Quinlan 1996 年的 C4.5）
 - 多层神经网络
 - 径向基函数网络
- 基学习器能否在加权示例上操作？
 - 在许多情况下，它们可以被修改为接受带有权重的示例
 - 通常，我们可以根据权重定义的分布对示例（带放回）进行抽样

- 损失函数:

$$\lambda(y, h(x)) \quad \text{例如 } I(y \neq h(x))$$

- 泛化误差:

$$L(h) = \mathbb{E}[\lambda(y, h(x))]$$

- 目标: 找到具有最小泛化误差的 h
- 提升的主要想法: 最小化经验误差 (模型在训练数据集上的错误率)

$$\hat{L}(h) = \frac{1}{N} \sum_{i=1}^N \lambda(y_i, h(x_i))$$

指数损失

- 经验损失:

$$\hat{L}(h) = \frac{1}{N} \sum_{i=1}^N \lambda(y_i, \hat{h}_m(x_i))$$

- 一种可能的经验损失度量是

$$\hat{L}(h) = \sum_{i=1}^n \exp\{-y_i \hat{h}_m(x_i)\}$$

指数损失

- 经验损失的可能度量形式: $\hat{h}_m(x) = \alpha_1 h(x; \theta_1) + \cdots + \alpha_m h(x; \theta_m)$

$$\begin{aligned}\hat{L}(h) &= \sum_{i=1}^n \exp\{-y_i \hat{h}_m(x_i)\} \\&= \sum_{i=1}^n \exp\{-y_i \hat{h}_{m-1}(x_i) - y_i \alpha_m h(x_i; \theta_m)\} \\&= \sum_{i=1}^n \exp\{-y_i \hat{h}_{m-1}(x_i)\} \exp\{-y_i \alpha_m h(x_i; \theta_m)\} \\&= \sum_{i=1}^n W_i^{m-1} \exp\{-y_i \alpha_m h(x_i; \theta_m)\} \quad \left(W_i^{m-1} = \exp\{-y_i \hat{h}_{m-1}(x_i)\}\right)\end{aligned}$$

- 基于 m 次迭代的组合分类器为下一个要添加的简单分类器定义了一个加权损失准则
- 每个训练样本按其“可分类性”加权, 该难度由我们到目前为止构建的分类

损失函数的线性化

- 我们可以简化新组件分类器的估计准则（假设 a 很小）

$$\exp\{-y_i a_m h(\mathbf{x}_i; \theta_m)\} \approx 1 - y_i a_m h(\mathbf{x}_i; \theta_m)$$

- 现在的经验损失

$$w_i^{m-1} = \exp\{-y_i h_{m-1}(x_i)\}$$

$$\begin{aligned}\sum \exp\{-y_i h_m(x_i)\} &\approx \sum w_i^{m-1} (1 - y_i a_m h(x_i; \theta_m)) \\ &\approx \sum w_i^{m-1} - a_m \sum w_i^{m-1} y_i h(x_i; \theta_m)\end{aligned}$$

- 我们可以选择一个新的成分分类器来优化这种加权一致性

一种可能的算法

- 在第 m 阶段，我们找到 θ^* 以最大化（或至少给出足够高的）加权一致性：

$$\sum_{i=1}^n W_i^{m-1} y_i h(\mathbf{x}_i; \theta_m^*)$$

- 每个样本根据先前 $m-1$ 个组合分类器下的“难度”加权
- 更“困难”的样本获得更多关注，因为它们主导总损失
- 然后我们回去找到与新分类器相关的“投票” α_m ，通过最小化原始加权（指数）损失

$$L(\hat{h}) = \sum_{i=1}^n W_i^{m-1} \exp\{-y_i \alpha_m h(\mathbf{x}_i; \theta_m)\}$$

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

AdaBoost 算法

- 在第 k 次迭代中，我们找到（任意）分类器 $h(\mathbf{x}; \theta_k^*)$ ，使得加权分类误差

$$\mathcal{E}_k = \frac{\sum_{i=1}^n W_i^{k-1} I(y_i \neq h(\mathbf{x}_i; \theta_k^*))}{\sum_{i=1}^n W_i^{k-1}}$$

优于随机猜测。

- 这应该“容易”——弱分类器
- 确定给新组件分类器分配多少“投票”：

$$\alpha_k = 0.5 \log \frac{1 - \mathcal{E}_k}{\mathcal{E}_k}$$

- 更强的分类器获得更多投票
- 更新训练样本权重：

$$W_i^k = W_i^{k-1} \exp\{-y_i \alpha_k h(\mathbf{x}_i; \theta_k)\}$$

- 经过 m 次提升迭代后的最终分类器由

$$\hat{h}(x) = \frac{\alpha_1 h(x; \theta_1) + \cdots + \alpha_m h(x; \theta_m)}{\alpha_1 + \cdots + \alpha_m}$$

的符号给出

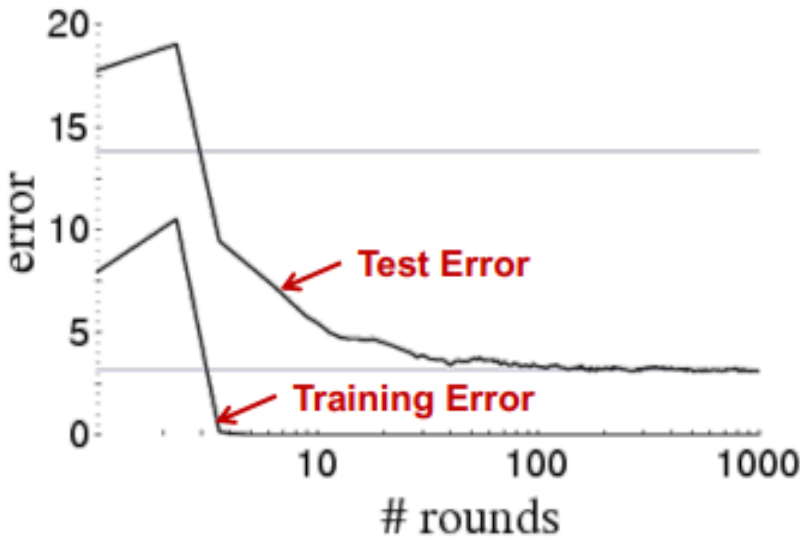
- 这里的投票被归一化以便使用

- 我们基本上推导出了一个提升算法，该算法按顺序添加新的组件分类器，每个分类器都在重加权训练示例上训练
- 每个组件分类器被呈现一个略有不同的问题
- AdaBoost 预备：
 - 我们使用训练示例上的归一化权重 W_i ，初始为均匀分布 ($W_i = 1/n$)
 - 权重反映了每个数据在最新分类器下的“困难程度”

AdaBoost: 总结

- 输入:
 - N 个示例 $S_N = \{(x_1, y_1), \dots, (x_N, y_N)\}$
 - 一个弱基学习器 $h = h(x; \theta)$
- 初始化: 所有 $i = 1..N$ 的权重相等 $w_i = 1/N$
- 迭代 $t = 1 \dots T$:
 - ① 根据加权示例集 (w_t, x) 训练基学习器, 得到假设 $h_t = h(x; \theta_t)$
 - ② 计算假设误差 $\mathcal{E}_t$
 - ③ 计算假设权重 α_t
 - ④ 更新下一次迭代的示例权重 w_{t+1}
- 输出: 最终假设为 h_t 的线性组合

提升结果——数字识别



Outline

- 1 集成方法
- 2 AdaBoost 算法
- 3 AdaBoost 的泛化特性
- 4 提升与逻辑回归
- 5 梯度提升

泛化性：VC 维

VC 维是衡量模型复杂度的一个指标，它描述了模型能够正确分类的最复杂的数据集的大小。

。对于 T 轮提升，其假设集定义为

$$\mathcal{F}_T = \left\{ \text{sgn} \left(\sum_{t=1}^T \alpha_t h_t \right) : \alpha_t \in \mathbb{R}, h_t \in \mathcal{H}, t \in [1, T] \right\}.$$

$\mathcal{F}_T$ 表示经过 T 轮提升后的 AdaBoost 算法的所有可能输出，符号函数 $\text{sgn}(\cdot)$ 是一个数学函数，它返回输入值的符号。在提升算法 (Boosting) 中， α_t 表示第 t 轮提升中基分类器 h_t 的权重， h_t 是从基分类器集合 $\mathcal{H}$ 中选出的分类器，而 T 是提升的总轮数。

AdaBoost 假设集的 VC 维的上界

$$\text{VCdim}(\mathcal{F}_T) \leq 2(d+1)(T+1) \log_2((T+1)e).$$

随着 T 的增加，AdaBoost 假设集的 VC 维也会增加，这意味着模型的复杂度在增加。模型过于复杂，以至于它不仅学到了数据中的有用信息，还学到了噪声和异常值。AdaBoost 可能会过拟合。

泛化误差界

化误差是指模型在真实数据上的错误率 $error_{\text{true}}(H)$ 与在训练数据上的错误率 $error_{\text{train}}(H)$ 之间的差异。

$$error_{\text{true}}(H) \leq error_{\text{train}}(H) + \tilde{O} \left(\sqrt{\frac{Td}{m}} \right)$$

其中 T 表示提升轮数, d 表示弱学习器的 VC 维, 衡量分类器复杂度, m 表示训练示例数量

- 当训练样本数量较少 (m) 或模型复杂度 (由 T 和 d 决定) 较高时, 这个项会较大, 导致泛化误差增加。

泛化误差界

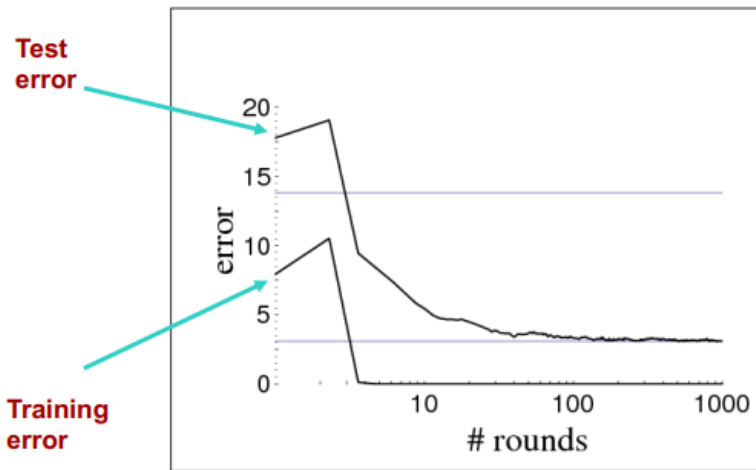
- 偏差 (bias): 模型预测结果与真实结果之间的差异。偏差大意味着模型过于简单, 无法捕捉数据的真实模式。
- 方差 (variance): 模型对训练数据的敏感程度。方差大意味着模型对训练数据的微小变化非常敏感, 可能导致过拟合。

	bias	variance	
tradeoff 	large	small	T small
	small	large	T large

- 当 T (提升轮数) 较小时, 偏差较大, 方差较小。——欠拟合
- 当 T 较大时, 偏差较小, 方差较大。——过拟合

- 理论认为提升在 T 很大时可能过拟合
- 实验中，在许多情况下，已经观察到 AdaBoost 的泛化误差随着提升轮数 T 的增加而减少

——我们需要更好的分析工具 (基于边界)



训练边界

- 当进行投票时，越多预测器一致，我们对预测越有信心
- 边界 (Margin) 是一个衡量模型对训练样本分类正确性的信心的指标。
- 对于样本 (x_i, y_i) , 边界:

$$\text{margin}_h(x_i, y_i) = y_i \left[\alpha_1 h(x_i; \theta_1) + \cdots + \alpha_m h(x_i; \theta_m) \right] / \sum_{t=1}^m \alpha_t$$

其中, $h(x_i; \theta_t)$ 是第 t 轮提升中使用的弱学习器, α_t 是对应的权重

- 间隔范围在 $[-1, 1]$, 对于所有被错误分类的样本, 边界值是负的。
- 连续的提升迭代会改进训练样本的多数投票或边界。这意味着随着提升算法的进行, 模型对训练数据的拟合会越来越好, 从而提高边界值。

对任意 γ ，泛化误差小于：

$$\Pr(\text{margin}_h(\mathbf{x}, y) \leq \gamma) + O\left(\sqrt{\frac{d}{m\gamma^2}}\right)$$

随着边界值的增加，模型的泛化误差会减少。

它不受 T 的限制!!!

这意味着即使增加提升轮数，只要边界值足够大，模型的泛化能力也不会受到影响。

总结

- 提升将弱学习器转化为强学习器
提升算法通过迭代地训练弱学习器，每个新的弱学习器都试图纠正前一个学习器的错误，从而逐步提高模型的整体性能。
- 通过渐近最小化经验误差工作
提升算法通过不断迭代，逐步减少模型在训练数据上的错误率，理论上可以无限接近于最小化这个错误率。
- 有效最大化组合假设的边界
提升算法通过优化组合假设的边界，使得模型不仅在训练数据上表现良好，而且在未见过的数据上也能保持良好的性能。

Outline

- 1 集成方法
- 2 AdaBoost 算法
- 3 AdaBoost 的泛化特性
- 4 提升与逻辑回归**
- 5 梯度提升

提升与逻辑回归

- 逻辑回归假设：

$$P(Y = 1 \mid x) = \frac{1}{1 + \exp(-f(x))}, \quad f(x) = w_0 + \sum_j w_j x_j$$

- 并试图最大化数据似然：

$$P(\mathcal{D} \mid f) = \prod_{i=1}^m \frac{1}{1 + \exp(-y_i f(x_i))}$$

- 等价于最小化对数损失

$$-\log P(\mathcal{D} \mid f) = \sum_{i=1}^m \ln(1 + \exp(-y_i f(x_i)))$$

提升与逻辑回归

逻辑回归等价于最小化对数损失

$$\sum_{i=1}^m \ln(1 + \exp(-y_i f(x_i))), \quad f(x) = w_0 + \sum_j w_j x_j$$

提升最小化的损失函数:

$$\frac{1}{m} \sum_{i=1}^m \exp(-y_i f(x_i)) = \prod_t Z_t, \quad f(x) = \sum_t \alpha_t h_t(x)$$

加权平均的弱学习器都是对 0/1 损失的平滑近似!



提升与逻辑回归

- 提升:

- 最小化指数损失

$$\sum_{i=1}^m \exp(-y_i f(x_i))$$

- 定义 $f(x) = \sum_t \alpha_t h_t(x)$, 其中 $h_t(x)$ 动态定义以适应数据 (非线性分类器)
- 权重在每个迭代 t 逐步学习

- 逻辑回归:

- 最小化对数损失

$$\sum_{i=1}^m \ln(1 + \exp(-y_i f(x_i)))$$

- 定义 $f(x) = \sum_j w_j x_j$, 其中 x_j 预定义特征 (线性分类器)
- 联合优化所有权重 $w_0, w_1, w_2, \dots$

硬决策与软决策

- 加权平均的弱学习器

$$f(x) = \sum_t \alpha_t h_t(x)$$

- 硬决策/预测标签:

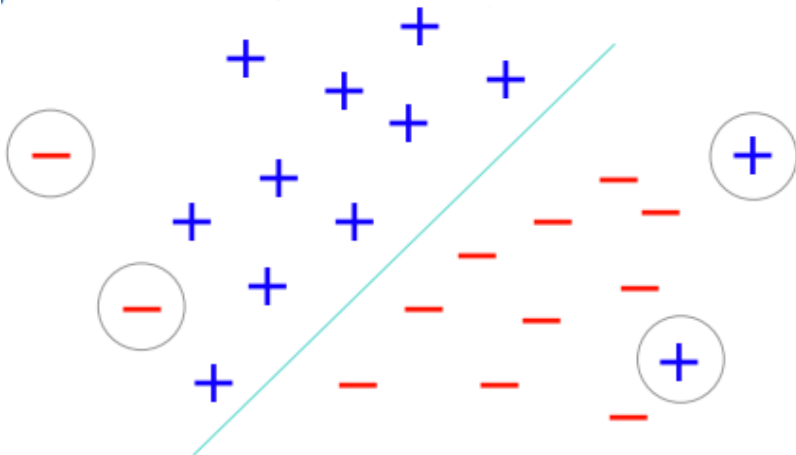
$$H(x) = \text{sign}(f(x))$$

- 软决策（基于与逻辑回归的类比）:

$$P(Y = 1 \mid X) = \frac{1}{1 + \exp(-f(x))}$$

异常值的影响

- 优点：可以识别异常值，因为它专注于难以分类的示例
- 缺点：过多异常值会显著降低分类性能，显著增加收敛时间
分配给更难分类的示例的分布权重随着提升轮数的增加而大幅增加。



AdaBoost 总结：优缺点

- 优点
简单，实施直接
每一轮提升的时间复杂度与样本量的函数相当有利。当使用决策残端时，每轮提升的时间复杂度以 $O(mN)$ 为单位。
- 缺点
需要选择参数 T 和基本分类器，并且在存在噪声的情况下性能较差。
在实践中， T 通常通过交叉验证来确定。

Outline

- 1 集成方法
- 2 AdaBoost 算法
- 3 AdaBoost 的泛化特性
- 4 提升与逻辑回归
- 5 梯度提升**

梯度提升

- 目标：找到非线性预测器 $\hat{h}(x) \in \mathcal{H}$ ，使得

$$\hat{h} = \arg \min_{h \in \mathcal{H}} L(h(X), Y)$$

- 梯度提升将 Adaboost（指数损失）推广到任意平滑损失函数
 - 平方损失（回归）

$$\mathcal{L}(h(X), Y) = \sum_{i=1}^n (h(\mathbf{x}_i) - y_i)^2$$

- 逻辑损失（分类）

$$\mathcal{L}(h(X), Y) = \sum_{i=1}^n \ln(1 + e^{-h(\mathbf{x}_i)y_i})$$

- 边界损失（排序）

$$\mathcal{L}(h(X), Y) = \sum_{(i, i') : y_{(i, i')} = 1} \max(0, 1 - (h(\mathbf{x}_i) - h(\mathbf{x}_{i'})))^2$$

优先选择项 i 而不是 j

梯度下降算法

对一般损失函数而言，往往每一步优化并不那么容易。梯度提升其基本原理是根据当前模型损失函数的负梯度信息来训练新加入的弱分类器，然后将训练好的弱分类器以累加的形式结合到现有模型中。

梯度下降算法的步骤如下：

- 1 确定当前位置的损失函数的梯度，对于 θ_t ，其梯度表达式如下：

$$\frac{\delta L(\theta)}{\delta \theta_t}$$

- 2 用步长 α 乘以损失函数的梯度，得到当前位置下降的距离，即：

$$\alpha \times \frac{\delta L(\theta)}{\delta \theta_t}$$

- 3 确定是否 θ_t 梯度下降的距离小于 ε ，如果小于 ε 则算法终止，当前的 θ_t 即为最终结果。否则进入步骤（4）。

- 4 更新 θ_t ，其更新表达式如下。更新完毕后继续转入步骤（1）。

$$\theta_{t+1} = \theta_t - \alpha \times \frac{\delta L(\theta)}{\delta \theta_t}$$

梯度提升决策树

- 采用决策树作为弱分类器的 Gradient Boosting 算法被称为 GBDT，有时又被称为 MART (Multiple Additive Regression Tree)。GBDT 中使用的决策树通常为 CART。
- 一个 J 叶节点的决策树可以视为输入空间的一个划分
- 以及

$$q: \mathbb{R}^d \rightarrow \{1, 2, \dots, J\}$$

- 与每个划分相关联的预测值 (权重)

$$w \in \mathbb{R}^J$$

- 将首先学习 q (树结构)，然后学习权重 w

常见的提升

Algorithm	Name	Loss	Derivative	f^*
L2Boosting	squared error	$\frac{1}{2}(y_i - f(x_i))^2$	$y_i - f(x_i)$	$E[y x_i]$
Gradient Boosting	absolute error	$ y_i - f(x_i) $	$\text{sgn}(y_i - f(x_i))$	$\text{median}(y x_i)$
AdaBoost	exponential error	$\exp(-y_i f(x_i))$	$-y_i \exp(-y_i f(x_i))$	$\frac{1}{2} \log \frac{1-e_k}{e_k}$
LogitBoost	logloss	$\log(1 + e^{-y_i f(x_i)})$	$y_i + \frac{1}{1+e^{y_i f(x_i)}}$	$\frac{1}{2} \log \frac{1+e^{y_i f(x_i)}}{1-e^{y_i f(x_i)}}$

Feedback welcome!

The End